Training Plan — Revision Run

Reinforcement Learning-based Autonomous Rendezvous for Docking of Reconfigurable Modular Robots in Unknown Environments

Prepared: 2026-05-13

Purpose. Operational plan for the new training run that supports the journal revision. This document fixes hyperparameters, the training matrix, file structure, and execution commands so that the experiments are reproducible and the resulting numbers flow directly into the revised paper.

Locked choices
• Hyperparameters: paper-stated — lr = 9×10^{-5}, ent_coef = 0.05, goal reward = +100.
• Scope: 15 primary runs + 25 reward-sensitivity runs = 40 runs total. Approximately 9 hours wall-clock on the RTX 5090. No N>5 stretch.
• All new code lives in new files; legacy scripts (`v*.py`, `rl_*.py`, `map_gen_*.py`) are left untouched.

What this run delivers for the revision

- Reproducibility: all reported numbers regenerated from a single command, with a saved `config.json` per run. Closes the gap between paper text and code.
- Uncertainty quantification: mean ± std across 3 seeds in every results table (answers R3 #15).
- Quantitative energy claim: total distance and max per-robot distance per config, with error bars (answers R2 #3).
- Reward sensitivity: 5 parameters × 5 levels (answers R3 #8 and #14, and supports R4 #1).
- Compute disclosure: wall-clock training and inference times reported per config (answers R3 #2 and #11).

1. Training matrix

1.1 Primary runs (15 total)

Config	N robots	Map size	Seeds	Steps	Notes
C1	2	20×20	{0,1,2}	100k	Smallest fleet
C2	3	20×20	{0,1,2}	100k	Matches Fig. 4(a)
C3	4	20×20	{0,1,2}	100k	Matches Fig. 4(b)
C4	5	20×20	{0,1,2}	100k	Matches Fig. 4(c)
C5	5	25×25	{0,1,2}	150k	Map-size scalability (Test Case 2)

Effort: ~20 min per 100k-step run on the 5090; C5 at ~30 min. Sub-total: **~5 hours**.

1.2 Reward sensitivity sweep (25 runs)

Five reward parameters × five perturbation levels, single seed each, 50k steps. Driven by the existing `reward_sensitivity.py` (paper hyperparameters already correct in that file).

Parameter	Paper default	Levels swept
area_decrease	+20	{+5, +10, +20, +40, +80}
area_increase	-0.5	{-2.0, -1.0, -0.5, -0.25, -0.1}
collide_obstacle	-5	{-20, -10, -5, -2, -1}
collide_robot	-5	{-20, -10, -5, -2, -1}
goal	+100	{+25, +50, +100, +200, +500}

Effort: 25 runs × ~10 min = **~4 hours**. Sensitivity sweep runs on N=4, 20×20 grid (the median primary config).

1.3 Total budget

Primary 5 h + sensitivity 4 h ≈ **9 hours wall-clock**. Add another 1-2 hours for evaluation rollouts (no training) once policies are saved. Whole run fits comfortably in a single overnight session.

2. Hyperparameters (locked, paper-stated)

Parameter	Value	Source
Algorithm	PPO (Stable-Baselines3)	Paper III-B
Policy	MultiInputPolicy	Matches v7.py
learning_rate	9e-5	Paper III-C.2
ent_coef	0.05	Paper III-C.1
n_steps	2048	SB3 default
batch_size	64	SB3 default
n_epochs	10	SB3 default
gamma	0.99	SB3 default; declare in paper MDP tuple
gae_lambda	0.95	SB3 default
clip_range	0.2	SB3 default
Device	cuda (RTX 5090, cu128 build)	Hardware
Total steps per run	100k (primary) / 50k (sensitivity) / 150k (see matrix)	SC5

2.1 Reward table (locked, paper Table III)

Event	Reward
Bounding-area decrease (below previous min, above threshold)	+20
Bounding-area decrease (below threshold = goal)	+100
Bounding-area increase	-0.5
Collision with obstacle	-5
Collision with another robot	-5

3. New files to create (no legacy modifications)

`env_revision.py`

Clean re-implementation of v7's RectangleReductionEnv using the gymnasium API. Rewards exactly per Table III. Per-robot distance counters exposed via the info dict. Deterministic seeding via gymnasium's seed() and reset(seed=...). Same observation/action spaces as v7 so the policy architecture remains comparable to the paper's network. Imports the connectivity-checked map generator from map_revision.py.

`map_revision.py`

Thin wrapper that calls map_gen_v5.py's existing logic but exposes seed and obstacle-density as arguments. Guarantees BFS connectivity of free cells before returning. Does not modify map_gen_v5.py.

`train_revision.py`

CLI driver. Arguments: --n-robots, --map-size, --seed, --steps, --tag. Writes model and config.json to logs_revision////. Uses paper hyperparameters as defaults. Records wall-clock training time and total environment steps in config.json.

`eval_revision.py`

Loads a saved policy and rolls out 50 episodes on each of the Test Case 1-5 maps with shared seeds. Logs total distance, max per-robot distance, success flag, steps to convergence, and mean inference time per step. Emits one row per (method, config, map, seed) into results.csv. Also runs the A* baseline (Section 5 below) on the identical maps.

`astar_baseline.py`

A* planner with three meeting-point heuristics: geometric median of robot positions, centroid of the largest connected free component of the known map, and current bounding-box center. Operates on the partially-known map (unknown cells treated as free). Re-plans every K = 5 steps as the map fills in. Terminates when the bounding area reaches the same threshold as PPO.

`plots_revision.py`

Reads results.csv and reward-sensitivity CSV, produces matplotlib figures: scaling curve with error bars (Fig. 6a revisited), PPO vs A* per-map comparison (new figure), sensitivity heatmap (new figure). All figures saved as both PDF and PNG for paper inclusion.

`run_all_revision.ps1`

PowerShell driver that sequentially launches all 40 training commands and then the eval rollouts. Logs to revision_run.log. Single command kicks off the whole run.

4. Reference commands

4.1 Single primary run

```
python train_revision.py --n-robots 4 --map-size 20 --seed 0 --steps 100000 --tag primary_v1
```

4.2 Whole primary matrix (PowerShell)

```
foreach ($n in 2,3,4,5) { foreach ($s in 0,1,2) { python train_revision.py --n-robots $n --map-size 20 --seed $s --steps 100000 --tag primary_v1 } } foreach ($s in 0,1,2) { python train_revision.py --n-robots 5 --map-size 25 --seed $s --steps 150000 --tag primary_v1 }
```

4.3 Sensitivity sweep

```
python reward_sensitivity.py --n-robots 4 --map-size 20 --steps 50000 --tag sens_v1
```

Note: `reward_sensitivity.py` already uses the paper hyperparameters; we may need to add the CLI flags shown above as a small touch-up.

4.4 Evaluation rollouts

```
python eval_revision.py --tag primary_v1 --episodes 50 --baseline astar
```

4.5 Plots and tables

```
python plots_revision.py --results-csv results.csv --out-dir figures_revision/
```

4.6 One-shot end-to-end

```
powershell -File run_all_revision.ps1
```

5. Evaluation protocol (apples-to-apples vs A*)

5.1 Test case maps

Re-use the five test-case maps from the original paper (Section IV-A through IV-E). Each map is loaded from a fixed seed so that PPO and A* see identical environments. For each map: 50 rollouts with episode seeds 0…49.

5.2 Metrics (logged per rollout)

- **total_distance** — sum of grid cells moved across the fleet (proxy for total energy).
- **max_distance** — max per-robot distance (proxy for the worst-loaded robot — your balance claim).
- **success** — bounding area reached the threshold before the step cap.
- **steps_to_converge** — if successful, number of environment steps to threshold.
- **inference_time_ms** — mean policy / planner step time, on CPU and on GPU.

5.3 Statistical reporting

- Aggregate over the three training seeds and 50 episode seeds: mean ± std.
- Paired Wilcoxon signed-rank test per map for PPO vs A* on each metric.
- Confidence intervals on the scaling curve (Fig. 6a revisited) via 1000-sample bootstrap.

5.4 A* baseline details

- Three meeting-point heuristics evaluated; report the strongest result for A* (steel-manning the baseline so the comparison is fair).
- Re-plan period K = 5 environment steps.
- Unknown cells optimistic (treated free) in the planner; report a conservative variant in the supplementary material if time allows.
- Same termination threshold as PPO: bounding area ≤ threshold.

6. Execution order

#	Step	Effort
1	Write env_revision.py and map_revision.py; unit-test with a 10-step rollout.	~3 h
2	Write train_revision.py; smoke-test with --steps 5000 --n-robots 2.	~2 h
3	Write astar_baseline.py and eval_revision.py; smoke-test on one saved policy.	~3 h
4	Touch up reward_sensitivity.py for new CLI flags (no logic change).	~1 h
5	Kick off run_all_revision.ps1; let it run overnight (~9 h).	overnight
6	Run eval rollouts (PPO and A*) on all saved policies.	~1.5 h
7	Generate figures and tables via plots_revision.py.	~1 h
8	Update the paper text (Section III MDP, Section IV-X sensitivity, Section IV-Y baseline comparison) with new numbers.	~1 day

Total: ~3 days of focused coding + an overnight training run + ~1 day of paper writing. The training itself is unattended.

6.1 Risks and mitigations

- **New PPO numbers differ from Fig. 4 curves in the paper.** Plan: regenerate Fig. 4 from the new TensorBoard logs; this is a few hours of work in plots_revision.py.
- **A* baseline wins on some maps.** Acceptable and honest. Report it; the PPO contribution is then framed as comparable performance without a target-cell heuristic.
- **Training is slower than estimated.** Drop the C5 25×25 config to 100k steps if needed; the smaller-map runs are non-negotiable.
- **Sensitivity sweep takes longer than 4 h.** Drop sweep steps from 50k to 30k; relative ranking is what matters, not absolute convergence.

End of plan.